Introduction

This path tracer implements three distinct materials: Lambertian for diffuse surfaces, metal for reflective surfaces, and dielectric for transparent objects. The geometric primitives include spheres, quads, triangles, and boxes. The system supports multiple texture types including solid colors, procedural checkers, Perlin noise, and image textures. The camera system implements both depth of field and motion blur effects. The renderer uses multi-threaded processing to produce high dynamic range (HDR) images.

Core Architecture

Rays store origin, direction, and time information, the last enabling motion blur effects. The camera uses stratified sampling to achieve better convergence rates compared to pure random sampling. Depth of field is implemented by sampling ray origins from a lens disk to approximate camera aperture effects. Motion blur is handled by sampling random ray times within the shutter interval. The recursive ray_color function accumulates light contributions from surface reflections up to a maximum bounce depth.

Materials and Lighting

All materials use polymorphic inheritance with scatter and emitted methods to enable extensible material definitions. Lambertian materials scatter rays in random directions calculated from the surface normal plus a random unit vector, simulating diffuse reflection. Metal materials implement reflective surfaces with optional fuzzing to simulate surface roughness. Dielectric materials implement both reflection and refraction using Snell's law for realistic glass and transparent object behavior.

The system supports multiple texture types including solid colors, procedural checkers, Perlin noise for organic patterns, and image textures loaded via the STB library with bilinear filtering for smooth interpolation.

Diffuse light materials emit constant radiance and function as area lights to illuminate scenes. Isotropic materials support uniform volumetric scattering, enabling fog and smoke effects.

Geometry

Spheres support both static and animated motion blur through linear interpolation of center positions. Ray-sphere intersection is computed using the quadratic formula to find ray-surface intersection points. Quads are defined by a corner point and two edge vectors that determine the rectangular surface. Triangles store vertex positions and compute surface normals using cross products of edge vectors. Boxes are constructed from six quads to create rectangular volumes.

All geometric primitives implement the hittable interface to provide a unified intersection testing mechanism. The `constant_medium` class wraps geometry to create volumetric regions with density-based scattering behavior.

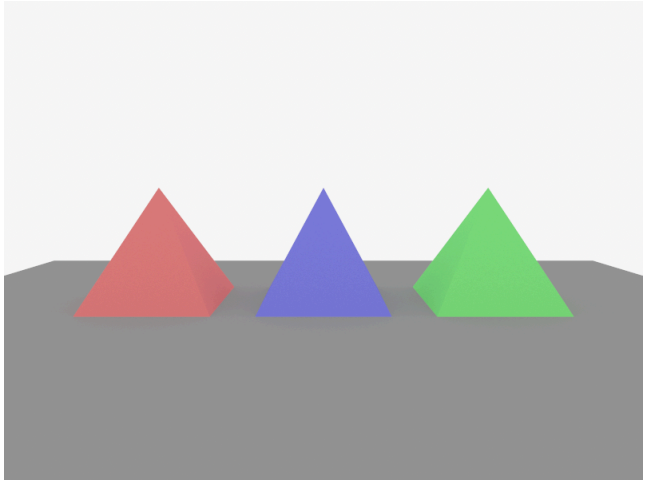


Figure 1: Three pyramids constructed from triangle primitives. 800x600, 1000 samples.

Acceleration

The Bounding Volume Hierarchy (BVH) organizes geometry into a binary tree structure for efficient spatial partitioning. Construction begins by computing bounding boxes for all primitives, sorting objects along the longest axis, and recursively splitting the list at the midpoint. Leaf nodes contain actual primitive references, while internal nodes store bounding boxes that encompass their children.

During traversal, the algorithm tests rays against bounding boxes and skips entire subtrees when rays miss their bounds. This approach reduces intersection test complexity from $O(n)$ to $O(\log n)$, which dramatically improves overall render times.

Volumetric Rendering

The `constant_medium` class wraps geometric boundaries and assigns density values to create volumetric regions. Rays are sampled for scattering distances using negative logarithmic sampling to model exponential attenuation through participating media. Isotropic materials scatter light uniformly in all directions, effectively simulating fog, smoke, and cloud-like effects. Multiple scattering events attenuate light as it travels through the medium, producing realistic translucent and diffuse appearance.

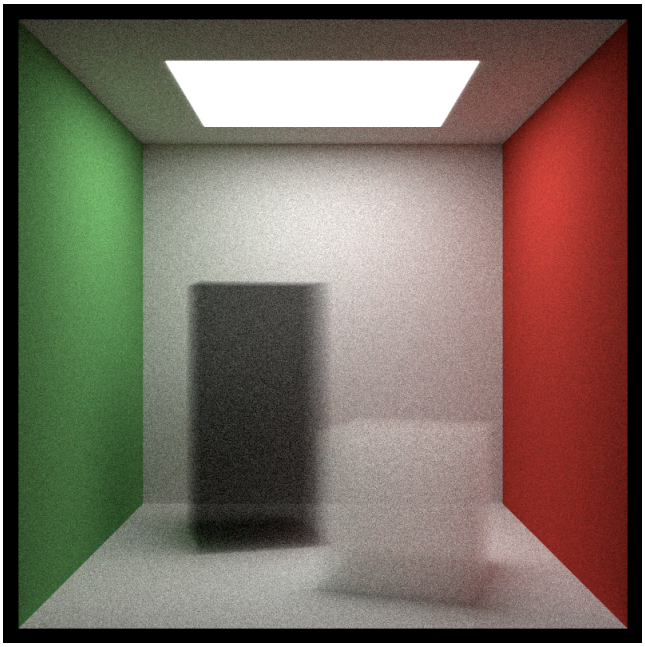


Figure 2: Cornell box with volumetric smoke showing color bleeding and light transport. 800x800, 400 samples.

Advanced Techniques

Depth of field is implemented by modeling the camera lens as a disk from which rays are sampled, with all rays directed toward the focal plane. Objects positioned at the focus distance remain sharp while objects at other distances exhibit blur effects.

Motion blur is achieved by sampling random ray times within the shutter interval, causing moving objects to interpolate their positions based on the sampled ray time. This produces natural motion trails in the output image.

The Monte Carlo path tracing algorithm traces rays recursively through the scene, scattering them according to material properties up to a maximum bounce depth. Multiple samples per pixel converge toward the true light transport solution, with variance decreasing proportionally to the square root of the sample count.

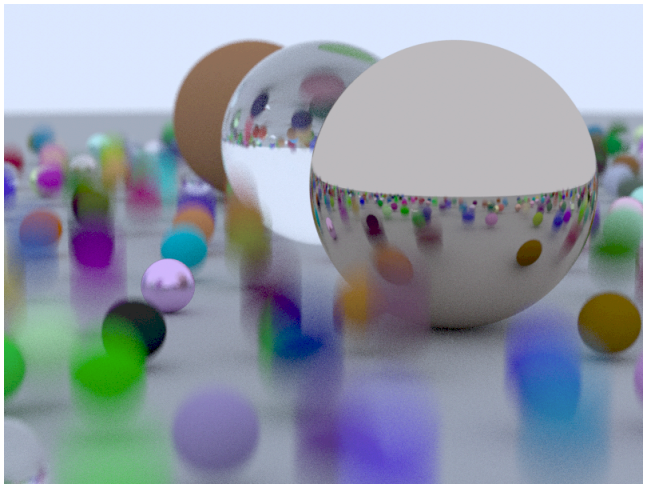


Figure 3: Motion blur with animated spheres and depth of field. 800x600, 100 samples.

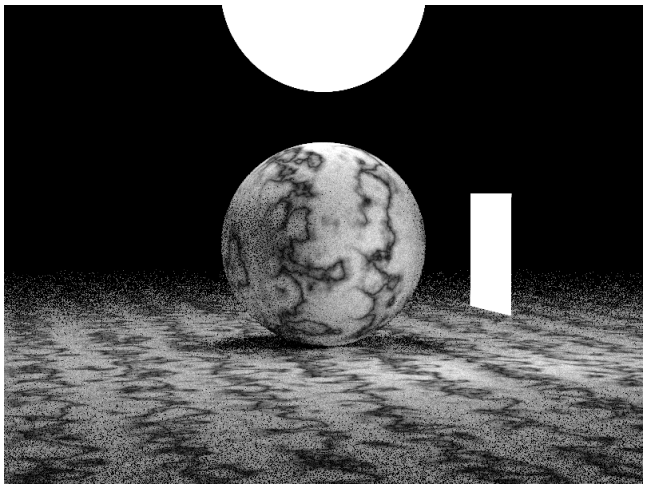


Figure 4: Area lights with Perlin noise spheres producing soft shadows. 800x600, 100 samples.

Performance

Multi-threaded rendering is the primary speedup mechanism for this raytracer. The renderer queries the system for the available number of CPU cores using `hardware_concurrency()` and spawns a worker thread for each core. Each worker thread independently renders a range of scanlines (rows) of the output image, computing all samples for its assigned pixels. Since pixels are independent—computing one pixel does not depend on neighboring pixels—there is zero inter-thread synchronization overhead.

A simple example demonstrates the speedup: on a quad-core CPU, four threads can process four different scanlines in parallel. If single-threaded rendering takes time t, four-threaded rendering takes approximately $\frac{t}{4}$. The parallelization scales nearly linearly because each thread performs identical work without lock contention or synchronization barriers. On an eight-core system, a 7-8x speedup is typical.

The BVH acceleration structure further reduces per-pixel cost by eliminating expensive linear scene traversal.

Transformations

The `translate` and `rotate_y` classes wrap hittable objects, transforming rays from world space to object space before performing intersection testing. After computing intersections, results are transformed back to world space coordinates. This deferred transformation approach preserves the original geometry data and avoids precision issues that can result from modifying vertex positions directly.

Output

The renderer produces output as Radiance HDR images using the STB library, storing floating-point RGB values for each pixel. This format preserves the full dynamic range of computed light values without clipping. Gamma correction is applied using a square root approximation to linearize color values. Output filenames are automatically timestamped to avoid overwriting previous renders.

Design

The template-based vec3 class provides type flexibility for different precision requirements.

The only external dependency is the STB library, which is used for image input and output operations. Physically-based materials with importance sampling provide good default results without requiring careful parameter tuning. The BVH construction uses median splits along the longest axis to create balanced trees.

Conclusion

This path tracer implements physically-based rendering following Shirley's framework. The system includes material models (Lambertian, metal, dielectric), BVH acceleration with $O(\log n)$ complexity, and diverse primitives. Advanced effects include volumetric media, motion blur, and depth of field.

Rendered outputs validate the implementation across motion-blur, cornell-smoke, simple-light, triangles, and showcase scenes. The system demonstrates that CPU-based Monte Carlo path tracing with BVH acceleration achieves photorealistic results within practical constraints.

The modular architecture separates geometric representations, material models, and light transport algorithms. Each pixel represents Monte Carlo integration of stochastic light paths, converging to physically plausible radiance estimates.

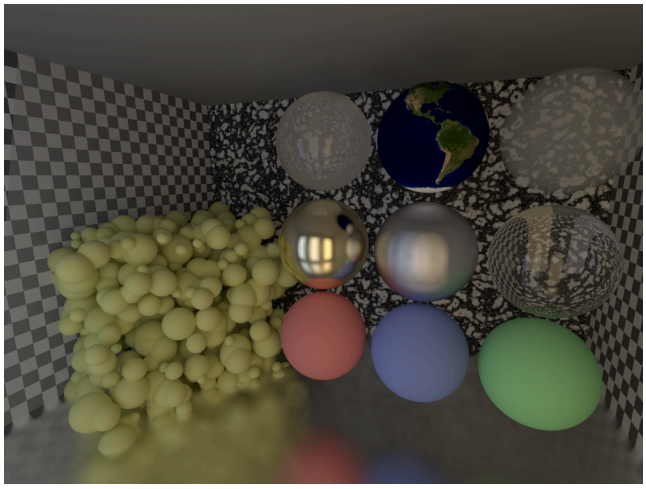


Figure 5: Comprehensive scene with 3x3 material grid, emissive bars, BVH-accelerated spheres, and volumetric regions. 1000x750, 10,000 samples.

References

This implementation is based on Peter Shirley's "Ray Tracing in One Weekend" series:

Shirley, P., Black, T. D., & Hollasch, S. (2025). *Ray Tracing in One Weekend.* Retrieved from https://raytracing.github.io/books/RayTracingInOneWeekend.html

Shirley, P., Black, T. D., & Hollasch, S. (2025). *Ray Tracing: The Next Week.* Retrieved from https://raytracing.github.io/books/RayTracingTheNextWeek.html

Shirley, P., Black, T. D., & Hollasch, S. (2025). *Ray Tracing: The Rest of Your Life.* Retrieved from https://raytracing.github.io/books/RayTracingTheRestOfYourLife.html